



University of Colombo, Sri Lanka

University of Colombo School of Computing

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

First Year Examination - Semester II - UCSC AY21 [held in March/April 2025]

SCS1309 Database Management Systems

(Two (2) Hours)

Answer ALL questions

235

Number of Pages = 12

Number of Questions = 4

To be completed by the candidate

Index Number:

--	--	--	--	--	--	--	--

Important Instructions to Candidates:

- I. Students should answer in the medium of English language only using the space provided in this question paper.
- II. Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
- III. Write your index number **CLEARLY** on each and every page of this Question paper.
- IV. This paper consists of 4 questions in 12 pages (including the Cover Page).
- V. **Answer ALL questions.**
- VI. Calculators and any electronic device capable of storing and retrieving text, including electronic dictionaries, smart watches and mobile phones **are not allowed**.
- VII. Do not tear off any part of this answer book. Under no circumstances may this book, used or unused, be removed from the Examination Hall by a candidate

To be completed by the examiners

Question No	Marks
1	
2	
3	
4	
Total	

Index No:

Question 1 – Total 40 Marks

Fashion Fusion is a trendy clothing retail business specializing in high-end fashion and accessories. The company operates multiple retail stores in various cities to serve its growing customer base. To streamline its operations, Fashion Fusion plans to implement a database system to effectively manage its retail stores, customers, staff, loyalty programs, products, and sales transactions.

Each retail store in the company is uniquely identified by a store_ID. A retail store also includes key attributes like name, location, and opening hours. Retail stores are classified as flagship stores, boutique outlets, and pop-up shops. Flagship stores have additional attributes, such as department count, while boutique outlets focus on exclusive item count. Pop-up shops, which offer limited-time designer collaborations, have the added attribute of event duration to track the time frame of events. Some retail stores may not fall into any of these categories, as Fashion Fusion occasionally experiments with different retail formats.

Fashion Fusion values its customers and operates a loyalty program to enhance their shopping experience. Each customer is identified by a customer_ID, and the system records attributes such as name, contact number, and email. Each customer can visit multiple retail stores, and each retail store can serve many customers. Additionally, customers can enroll in a loyalty program, which is uniquely identified by a program_ID. Each program has a program name and program description. A customer can only join one loyalty program at a time, but a single program can have many enrolled customers. The database needs to track the join date for each customer and their current points accumulated through their participation in the program.

Fashion Fusion employs staff members in various roles across its retail stores. Each staff member is uniquely identified by a staff_ID and includes attributes such as name and contact number. Each staff member is assigned to one retail store. Each retail store is managed by a staff member, and the database should record the start date of that staff member's management role in the retail store.

Fashion Fusion stocks various products in its retail stores. Each product is uniquely identified by a product_ID and includes key attributes such as name, category, and price. Products are stocked across multiple retail stores, and each retail store tracks the availability of each product.

Fashion Fusion tracks all customer purchases through sales transactions. Each transaction is identified by a unique transaction_ID and includes the date, time and the total value of the purchase. A single transaction is associated with only one customer, but a customer may have multiple transactions over time. Additionally, a transaction can involve multiple products, with each product's quantity recorded in the transaction.

- (a) Draw an EER diagram showing the necessary entities, attributes along with the primary key, and relationships for the above scenario. Specify the cardinality and participation constraints for each relationship set. Clearly state all your assumptions, if any.

[20 Marks]

Index No:

Index No:

- (b) Map the EER diagram developed in (a) above to a relational data model. Underline the primary key of each relation and explicitly indicate any foreign keys by marking them with (FK).

[20 Marks]

Question 2 – Total 10 Marks

- (a). Consider the relations given below and write appropriate relational algebra expressions.
Student(SID, Name, Age, Department) [1 X 5 = 5 Marks]

Course(CID, Title, Credits)

Enrollment(SID, CID, Grade)

- 1) A relational algebra expression to find students who are older than 20.

- 2) A relational algebra expression to display only the names and departments of students.

- 3) A relational algebra expression to generate all possible pairs of students and courses.

- 4) A relational algebra expression to retrieve students along with their enrolled courses.

- 5) A relational algebra expression to retrieve all distinct student IDs who have enrolled in at least one course.

- (b). Functional dependency identification helps in designing logically structured schemas.

- 1) Explain what you mean by the term functional dependency. [1 mark]

- 2) For each of the following statements, determine whether it is True or False. Regardless of the answer, you must provide a proper justification to explain why the statement is true or false. [2 X 2 = 4 Marks]

- (i) The functional dependency Student_ID $\rightarrow$ Student_Name means that multiple students can have the same name.

- (ii) If $A \rightarrow B$, then $B \rightarrow A$ must also be true.

Index No:

Question 3 – Total 20 Marks

A university database maintains students' course enrollments using a single table called **Student_Course** as follows.

S_ID	S_Name	C_ID	C_Name	Instructor_ID	Instructor_Name	Instructor_Office
101	Alice	CSE101	Database Systems	301	Dr. Smith	Room 301
102	Bob	CSE102	Data Structures	302	Dr. Brown	Room 302
101	Alice	CSE103	Operating Systems	303	Dr. Clark	Room 303
103	Charlie	CSE101	Database Systems	301	Dr. Smith	Room 301
102	Bob	CSE101	Database Systems	301	Dr. Smith	Room 301

(a). Explain the **three (03)** types of update anomalies using examples from the above table.

[6 marks]

(b). Determine the highest normal form satisfied by the above table. Justify your answer.

[4 marks]

Index No:

- (c). Convert the table into the third Normal Form (3NF). Indicate the steps, and tables at each step, including the keys and any assumptions made. **[10 Marks]**

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

52

Question 4 – Total 30 Marks

- (a). You are managing an online bookstore and have the following two tables. Based on it **write SQL statements for the questions given below.**

Book Table - Stores information about the books available for sale.

Column Name	Data Type	Description
bookID	INT	Unique identifier for each book.
title	VARCHAR(100)	Title of the book, Cannot be null.
author	VARCHAR(50)	The name of the author, Cannot be null.
price	DECIMAL(10,2)	The price of the book, Cannot be null.
stock	INT	The number of copies available, Cannot be null.
category	VARCHAR(30)	The genre or category of the book, Cannot be null.
publishedDate	DATE	The date when the book was published, Cannot be null.

Order Table - Stores customer purchases.

Column Name	Data Type	Description
orderID	INT	Unique identifier for each order.
bookID	INT	The book that was ordered, refers to Book(bookID) and when the parent is deleted child should be deleted as well, Cannot be null.
customerName	VARCHAR(50)	Customer's name, who placed the order, Cannot be null.
quantity	INT	Number of copies ordered, Cannot be null.
orderDate	DATE	Date when the order was placed, Cannot be null, Default: Current Date

Index No:

1. Create the **Order** table.

[6 Marks]

2. List all books published after the year 2015, displaying title, author, and publishedDate.

[1 Mark]

3. Get all books that belong to the "Fiction" category and have stock greater than 10.

[2 Marks]

4. Find the total number of books available in stock across all categories. **[2 Marks]**

5. Display the top 5 most recently published books.

[2 Marks]

Index No:

6. Write a **trigger** that automatically **updates the stock in the Book table** when a new order is placed in the Order table. **[3 Marks]**

7. Create a **stored procedure** that **returns all books** in a given category (passed as a parameter) and write a statement to show, how you call it. **[4 Marks]**

Index No:

8. Retrieve a list of customers and the books they ordered, including customerName, title, and quantity. [3 Marks]

- (b). Consider the following Scenario. [7 Marks]

Step 1: Creating the Product table and inserting data

Table structure used:

Column Name	Data Type	Description
productID	VARCHAR(10)	Unique identifier for each product
productName	VARCHAR(50)	Name of the product, Cannot be null
price	DECIMAL(10, 2)	Price of the product, Cannot be null
category	VARCHAR(20)	Category the product belongs to, Cannot be null

Row Inserted:

productID	productName	price	category
P101	Smartphone	1200.00	Electronics

Step 2: Creating Views

```
CREATE VIEW MidRangeProducts AS
SELECT * FROM Product
WHERE price > 900;
```

```
CREATE VIEW HighEndProducts AS
SELECT * FROM MidRangeProducts
WHERE price > 1000
WITH LOCAL CHECK OPTION;
```

Index No:

Next, a third view, PremiumProducts, will be created in two different ways:

- Using **CASCADED CHECK OPTION**
CREATE VIEW PremiumProducts AS
SELECT * FROM HighEndProducts
WHERE category = 'Electronics'
WITH **CASCADED CHECK OPTION**;
- Using **LOCAL CHECK OPTION**
CREATE VIEW PremiumProducts AS
SELECT * FROM HighEndProducts
WHERE category = 'Electronics'
WITH **LOCAL CHECK OPTION**;

Step 3: Executing Updates on the Views

The table below lists different UPDATE statements (Executed separately, not in a single run.) applied to PremiumProducts, HighEndProducts, and MidRangeProducts. Determine the execution result (State “**SUCCESS**” or “**FAIL**”) for each statement under the two different scenarios of ‘PremiumProducts’ view creation.

SQL Statement	CASCADED CHECK OPTION	LOCAL CHECK OPTION
UPDATE PremiumProducts SET price = 800 WHERE productID = 'P101';		
UPDATE PremiumProducts SET price = 950 WHERE productID = 'P101';		
UPDATE PremiumProducts SET price = 1050 WHERE productID = 'P101';		
UPDATE PremiumProducts SET category = 'Appliances' WHERE productID = 'P101';		
UPDATE MidRangeProducts SET price = 950 WHERE productID = 'P101';		
UPDATE HighEndProducts SET price = 950 WHERE productID = 'P101';		
UPDATE HighEndProducts SET price = 1000 WHERE productID = P101;		

*****THE END*****